

Assignment 2: Coding Basics

Josette McLean

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.

```
seq(1,55, 5)
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
seq1 <- seq(1,55,5)
```

2. Compute the mean and median of this sequence.

```
mean(seq1)
```

```
## [1] 26
```

```
median(seq1)
```

```
## [1] 26
```

3. Ask R to determine whether the mean is greater than the median.

```
mean(seq1)>median(seq1)
```

```
## [1] FALSE
```

4. Insert comments in your code to describe what you are doing.

```
#1. I generated a sequence of numbers from 1 to 55, increasing by 5, and named it seq1  
#2. Calculated the mean and median of seq1  
#3. Assessed if the mean was greater than median, it was not
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).

```
student_names <- c("Josette", "John", "Jane", "Joseph") #This is a character vector  
test_scores <- c(80, 100, 75, 50) #This is a integer vector  
on_scholarship <- c(TRUE, FALSE, FALSE, TRUE) #This is a logic vector
```

6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.

```
student_information <- data.frame(  
  student_names = student_names,  
  test_score = test_scores,  
  on_scholarship = on_scholarship  
)
```

8. Label the columns of your data frame with informative titles.

```
student_information <- data.frame(  
  student_names = student_names,  
  test_score = test_scores,  
  on_scholarship = on_scholarship  
)
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A matrix can only contain data of the same type, so it would not have been able to store each of these vectors of different data types, that is what makes this data frame different, because it can contain a variety of vectors with different data types.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".

```
evaluate_score <- function(score) {  
  if (score > 50) {  
    print("Pass")  
  } else {  
    print("Fail")  
  }  
}  
  
evaluate_score(75)
```

```
## [1] "Pass"
```

```
evaluate_score(40)
```

```
## [1] "Fail"
```

11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.

```
evaluate_score2 <- function(score) {  
  result <- ifelse(score > 50, "Pass", "Fail")  
  return(result)  
}  
  
evaluate_score2(75)
```

```
## [1] "Pass"
```

```
evaluate_score2(40)
```

```
## [1] "Fail"
```

12. Run both functions using the value 52.5 as the input

```
evaluate_score <- function(score) {  
  if (score > 50) {  
    print("Pass")  
  } else {  
    print("Fail")  
  }  
}  
  
evaluate_score(52.5)
```

```
## [1] "Pass"
```

```

evaluate_score2 <- function(score) {
  result <- ifelse(score > 50, "Pass", "Fail")
  return(result)
}

evaluate_score2(52.5)

```

```
## [1] "Pass"
```

13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```

test_scores <- c(80, 100, 75, 50)
#First way
evaluate_score <- function(score) {
  if (score > 50) {
    print("Pass")
  } else {
    print("Fail")
  }
}

```

```

# Second way
#evaluate_score2 <- function(score) {
#  # result <- ifelse(score > 50, "Pass", "Fail")
#  # return(result)
#}

#evaluate_score(test_scores)
#evaluate_score2(test_scores)

```

#10. Create a function using if...else

#11. Create a function using ifelse()

#12a. Run the first function with the value 52.5

#12b. Run the second function with the value 52.5

#13a. Run the first function with the vector of test scores

#13b. Run the second function with the vector of test scores

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The ‘`if ... else`’ worked while the `ifelse` did not work. I might have done something wrong because logically it should have been the other way around. This is because `if ... else` only checks one value at a time, so when I gave it a whole vector of test scores, it only looked at the first score and ignored the rest. On the other hand, `ifelse()` is vectorized, which means it can check every score in the vector, one by one, and return a result for each. That’s why `ifelse()` worked for the whole set of test scores. That what should have happened.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)